TaskTracker

Planejamento Lógico e Arquitetural — Fase 1

Bootcamp II — Engenharia de Software

Estudante	Sara Menezes Macedo
Curso	Ciência de Dados
Projeto	TaskTracker — gerenciador de tarefas em linha de comando (CLI)
Linguagem prevista	Python 3

1. Descrição do problema

Pessoas que organizam estudos e trabalho precisam registrar tarefas, saber quais estão pendentes e qual a importância e o prazo de cada uma. O TaskTracker é uma aplicação de terminal que resolve o núcleo desse problema: **cadastrar** tarefas com dados validados e **visualizar** o que já foi cadastrado, em um menu que permanece ativo até o usuário decidir sair.

Objetivo desta fase: planejar a solução antes de programar, definindo a decomposição do problema, a estrutura de dados, as regras de negócio, o algoritmo em português e o mapeamento do projeto para Git/GitHub.

2. Decomposição do problema

O problema foi dividido em partes menores e independentes, cada uma com uma responsabilidade:

Módulo	Responsabilidade	Entrada	Saída
Menu principal	Exibir opções e repetir até o usuário sair	Opção digitada (1, 2 ou 3)	Chamada à funcionalidade escolhida
Cadastro de tarefa	Coletar e validar os dados e guardar a tarefa na lista	Título, descrição, prioridade, data limite	Nova tarefa na lista; mensagem de sucesso
Validação	Aplicar as regras de negócio e repetir a pergunta se inválido	Texto digitado	Valor válido ou mensagem de erro
Visualização	Listar as tarefas ou avisar que não há registros	Lista de tarefas	Tarefas formatadas no terminal

3. Arquitetura de dados

As tarefas ficam em memória, em uma **lista** de **dicionários**. Cada dicionário representa uma tarefa:

Campo	Tipo	Obrigatório	Regra
titulo	Texto	Sim	Não pode ser vazio nem conter apenas espaços
descricao	Texto	Não	Texto livre; se vazia, grava “Sem descrição”

Campo	Tipo	Obrigatório	Regra
prioridade	Texto	Sim	Somente Alta, Média (ou Media) e Baixa
data_limite	Texto	Não	Formato DD/MM/AAAA e data existente; se vazia, “Não informada”
status	Texto	Automático	Sempre “Pendente” no cadastro

Exemplo de uma tarefa:

```
{
  "titulo": "Estudar para a prova de Lógica",
  "descricao": "Revisar variáveis e laços",
  "prioridade": "Alta",
  "data_limite": "20/10/2026",
  "status": "Pendente"
}
```

4. Regras de negócio

- **RN1 — Título obrigatório:** entradas vazias ou só com espaços são rejeitadas e o sistema pergunta novamente.
- **RN2 — Prioridade estrita:** apenas Alta, Média/Media e Baixa (sem diferenciar maiúsculas de minúsculas); qualquer outra entrada gera erro e repetição da pergunta.
- **RN3 — Data limite:** opcional; se informada, deve seguir DD/MM/AAAA e ser uma data real.
- **RN4 — Status inicial:** toda tarefa nova é criada como “Pendente”, sem intervenção do usuário.
- **RN5 — Lista vazia:** se não houver tarefas, a visualização exibe mensagem clara de que não existem registros.
- **RN6 — Menu contínuo:** opções inválidas no menu geram aviso e o menu é exibido de novo.

5. Algoritmo em português (pseudocódigo)

5.1 Programa principal

```
INICIO
  tarefas <- lista vazia
  REPITA
    EXIBIR menu (1-Cadastrar, 2-Visualizar, 3-Sair)
    LER opcao
    SE opcao = "1" ENTAO  cadastrar_tarefa(tarefas)
    SENAO SE opcao = "2" ENTAO  visualizar_tarefas(tarefas)
    SENAO SE opcao = "3" ENTAO  EXIBIR "Encerrando..." ; SAIR DO LACO
    SENAO  EXIBIR "Opcao invalida. Escolha 1, 2 ou 3."
    FIM SE
  ATE o usuario escolher sair
FIM
```

5.2 Cadastrar tarefa

```
FUNCAO cadastrar_tarefa(tarefas)
  titulo <- pedir_titulo()
  LER descricao
  SE descricao vazia ENTAO descricao <- "Sem descricao"
  prioridade <- pedir_prioridade()
  data_limite <- pedir_data_limite()
  tarefa <- { titulo, descricao, prioridade, data_limite,
```

```
              status: "Pendente" }
  ADICIONAR tarefa AO FINAL de tarefas
  EXIBIR "Tarefa cadastrada com sucesso!"
FIM FUNCAO
```

5.3 Validações (repetem até o dado ser válido)

```
FUNCAO pedir_titulo()
  REPITA
    LER titulo
    SE tamanho(remover_espacos(titulo)) > 0 ENTAO RETORNAR titulo
    EXIBIR "Erro: o titulo nao pode ficar vazio."
  ATE valido

FUNCAO pedir_prioridade()
  REPITA
    LER texto
    p <- converter texto para minusculas e remover espacos
    SE p = "alta" ENTAO RETORNAR "Alta"
    SE p = "media" OU p = "média" ENTAO RETORNAR "Média"
    SE p = "baixa" ENTAO RETORNAR "Baixa"
    EXIBIR "Erro: prioridade invalida."
  ATE valido

FUNCAO pedir_data_limite()
  REPITA
    LER texto
    SE texto vazio ENTAO RETORNAR "Nao informada"
    SE texto e uma data valida no formato DD/MM/AAAA ENTAO RETORNAR texto
    EXIBIR "Erro: use o formato DD/MM/AAAA."
  ATE valido
```

5.4 Visualizar tarefas

```
FUNCAO visualizar_tarefas(tarefas)
  SE quantidade(tarefas) = 0 ENTAO
    EXIBIR "Nenhuma tarefa cadastrada no momento."
    RETORNAR
  PARA CADA tarefa EM tarefas, com numero iniciando em 1, FACA
    EXIBIR numero, titulo, descricao, prioridade, data_limite, status
  FIM PARA
  EXIBIR "Total de tarefas: " + quantidade(tarefas)
FIM FUNCAO
```

6. Casos de teste planejados

#	Cenário	Entrada	Resultado esperado
1	Visualizar sem tarefas	Opção 2 ao iniciar	Mensagem de que não há registros
2	Título vazio	Enter ou apenas espaços	Erro e nova pergunta
3	Prioridade inválida	“urgente”	Erro e nova pergunta
4	Prioridade sem acento	“media”	Aceita e grava “Média”
5	Data inválida	“31/02/2026”	Erro e nova pergunta
6	Cadastro completo	Dados válidos	Tarefa criada com status Pendente
7	Opção de menu inválida	“9”	Aviso e menu exibido de novo

7. Mapeamento para Git e GitHub

7.1 Estrutura do repositório

```
tasktracker-projeto/
|-- README.md                  # documentação e instruções de uso
|-- .gitignore                 # ignora __pycache__/, *.pyc e .venv/
|-- docs/
|   `-- planejamento_logico.pdf  # este documento (Fase 1)
`-- src/
    `-- main.py                # código-fonte em Python
```

7.2 Plano de commits

Etapa	Comando / ação	Conteúdo
1	git init	Criar o repositório local e a estrutura de pastas
2	git commit (inicial)	README.md, .gitignore e docs/planejamento_logico.pdf
3	git commit (desenvolvimento)	src/main.py com menu e loop principal
4	git commit (regras)	Funções de validação de título, prioridade e data
5	git push -u origin main	Publicar no GitHub e manter o histórico rastreável

Cada commit representa uma evolução coerente do projeto, o que permite acompanhar o desenvolvimento e voltar a versões anteriores se necessário.